

Few main things that we learnt

Delta Lake

3 major pillars of data engineering

- Lakeflow Connect
- Lakeflow Declarative Pipelines
- Lakeflow Jobs

All these things we learnt along with Unity catalog

Now we have to learn about how your code is deployed..

Databricks Asset Bundles (DAB's)

DAB's heavily uses Databricks CLI

so lets first learn Databricks CLI

Databricks CLI is a wrapper on top of Databricks Rest API's.

Databricks CLI you can create your catalog, create schema, list catalogs..

Databricks workspace..

Databricks Account console - metastores, workspaces, users...

using Databricks CLI you can interact with both

- Databricks Workspace
- Databricks account

macOs: Homebrew, curl

Windows: WinGet

```
curl -fsSL https://raw.githubusercontent.com/databricks/setup-cli/main/install.sh |  
sudo sh
```

Installed Databricks CLI v0.282.0 at /usr/local/bin/databricks.

Now we will set up authentication between the databricks CLI and your

databricks workspace.

there are 2 ways of authentication

- U2M (your credentials)
- M2M (service principal)

U2M (your credentials)

=====

with U2M authentication, you log in interactively and the CLI manages short lived tokens on your behalf. token expire in under an hour, this reduces the risk of token being accidentally exposed.

databricks auth login --host <workspace-url>

databricks auth login --host
<https://adb-2361508760315198.18.azuredatabricks.net>

to check all the profiles
databricks auth profiles

.databrickscfg

cat .databrickscfg

; The profile defined in the DEFAULT section is to be used as a fallback when no profile is explicitly specified.
[DEFAULT]

[sumit]

host = <https://adb-2361508760315198.18.azuredatabricks.net>
auth_type = databricks-cli

databricks auth login --host
<https://adb-7405619447258681.1.azuredatabricks.net>

<https://accounts.azuredatabricks.net/>

U2M authentication

M2M authentication (service principal)

- workspace
- accounts console

user id :

sumit.trendytech03_outlook.com#EXT#@sumittrendytech03outlook.onmicrosoft.com

password:

how to create a user:

kapil@sumittrendytech03outlook.onmicrosoft.com

<https://adb-7405619447258681.1.azuredatabricks.net>

how to add a new user

entra id

synced it in our databricks account

group - added the user

permissions for the workspace

databricks workspace - gave privileges (SQL / Data Engineering)

Service Principal

go to azure portal -> app registrations

dab-learning-sp

display name: dab-learning-sp

application id: 1096594c-dec0-413a-86d3-f47ea4a737b3

=====

secret : dosefa36c9efa89984c4e58d19c663ee475d

client id: 1096594c-dec0-413a-86d3-f47ea4a737b3

=====

M2M authentication

vi .databrickscfg

For account-level commands

```
[automation-account-sp]
host      = https://accounts.azuredatabricks.net
account_id = 5d0145a3-793e-4127-b499-1fe6bff6b93c
client_id  = 1096594c-dec0-413a-86d3-f47ea4a737b3
client_secret = dosefa36c9efa89984c4e58d19c663ee475d
```

For workspace-level commands

```
[automation-workspace-sp]
host      = https://adb-7405619447258681.1.azuredatabricks.net
client_id  = 1096594c-dec0-413a-86d3-f47ea4a737b3
client_secret = dosefa36c9efa89984c4e58d19c663ee475d
```

=====

Databricks CLI

Service Principal

Databricks git folders (Databricks repos)

Github

Azure Devops

a new databricks workspace - trendytech-dab-demo-dev-ws

azure devops

<https://aex.dev.azure.com>

=> create a organization

=> create a project

=> create a repo (dab-demo)

integrate this azure devops repo with my databricks

url:

https://sumittrendytech03@dev.azure.com/sumittrendytech03/trendytech-retail/_git/dab-demo

username: sumit.trendytech03

password:

C2T97WynUYzjDf7bSY2Z0vi3CPfuRpCstp7sM7fMBIMoJCVcyj8fJQQJ99CAAC
AAAAAAAAAAAAASAZDOYsB9

after linking your remote azure devops account...

we need to create a git folder

main branch has 10 files

feature-101

I would want the same code as in the main branch

feature-101 (10 files)

2 new files (12 files)

main - 10 files

feature - 12 files

=====

Databricks Asset Bundles (DAB's)

Bundle you will create consisting all your assets

Bundles provide you a way to include metadata alongside your project source files.

Job

bronze -> silver -> gold

bundle => source code + metadata

A bundle includes the following parts:

- source code files, such as notebooks and python files
- includes metadata, definitions and settings for databricks resources, such as lakeflow jobs, lakeflow declarative pipelines
- required cloud infrastructure
- unit test and integration test

Bundle Metadata is defined using YAML files

its a more readable thing than json.

databricks asset bundle

- on your local (pycharm or Visual studio code)
- databricks UI

<https://code.visualstudio.com/>

trendytech-dab-demo-dev-ws

<https://adb-7405612598005272.12.azuredatabricks.net>

databricks auth login <https://adb-7405612598005272.12.azuredatabricks.net>

sumitdev

if I want I can define the bundle from scratch..

you go with the bundle structure

main folders and files are

.databricks
- resources
- src
databricks.yml

- resources (job.yml)
- src (source code)
databricks.yml (parent configuration for the bundle)

.bundle -> trendytech_dab_project -> dev -> files

resources (yml files)
src (source code)
databricks.yml

=====

Job

Pipeline

```
resources:
  jobs:
    orders_job:
      name: orders_job
      tasks:
        - task_key: bronze_ingestion
          notebook_task:
            notebook_path:
/Workspace/Users/sumit.trendytech03@outlook.com/orders_bronze
            source: WORKSPACE
        - task_key: silver_cleaning
          depends_on:
            - task_key: bronze_ingestion
          notebook_task:
            notebook_path:
/Workspace/Users/sumit.trendytech03@outlook.com/orders_silver
            source: WORKSPACE
      queue:
        enabled: true
        performance_target: PERFORMANCE_OPTIMIZED
```

`databricks bundle validate -t dev`

`/Workspace/Users/sumit.trendytech03@outlook.com/.bundle/trendytech_dab_project/dev/files/src/orders_bronze`

`databricks bundle validate --target dev -p sumitdev`

`databricks bundle summary --target dev -p sumitdev`

`databricks bundle deploy --target dev -p sumitdev`

`databricks auth login https://adb-7405615339379447.7.azuredatabricks.net`

`databricks bundle validate --target prod -p sumitprod`

`databricks bundle summary --target prod -p sumitprod`


```
databricks bundle deploy --target prod -p sumitprod
```

=====

VScode

- Notebooks
- job (resources, src, databricks.yml)

```
databricks bundle init
```

```
databricks bundle validate --target dev
```

```
databricks bundle summary --target dev
```

```
databricks bundle deploy --target dev
```

```
/Workspace/Users/sumit.trendytech03@outlook.com/dab-project/dab_new_project/src/orders_bronze
```

```
/Workspace/Users/sumit.trendytech03@outlook.com/.bundle/dab_new_project/dev/files/src/orders_bronze
```

=====

job

bronze

- orders

silver

- orders

dev env

dev_bronze

- orders

dev_silver
- orders

prod env

prod_bronze

prod_silver

resources:

jobs:

orders_job:

name: orders_job

tasks:

- task_key: bronze_ingestion

notebook_task:

notebook_path: ../../src/orders_bronze.ipynb

base_parameters:

environment_name: dev

source: WORKSPACE

- task_key: silver_cleaning

depends_on:

- task_key: bronze_ingestion

notebook_task:

notebook_path: ../../src/orders_silver.ipynb

base_parameters:

environment_name: dev

source: WORKSPACE

queue:

enabled: true



performance_target: PERFORMANCE_OPTIMIZED

we saw how to create a dab project on visual studio code

notebooks, pipelines, jobs,

how to make things dynamic using variables

how to create a dab project in databricks UI

